Self-Organized Criticality: Sandpile Model on Undirected Graph, Influence of Topology and Dissipation Rules on Avalanches

1. Introduction

In everyday life, person's outburst is rarely caused by a single large event. Rather, the tension builds up from small frustrations over time. When certain threshold is reached, even a minor additional stress can trigger an outburst. That redistributes the tension to others in their social circle. This may in turn cause further reactions, leading to a cascade of reach and size which are difficult to predict. A local event may therefore propagate through a network, depending on individual thresholds and the shape of the network.

This may seem irregular or unpredictable, but many complex systems exhibit stable statistical properties when the number of observation is large. This phenomenon is explained by the law of large numbers [1] which applies, for example, to repeated coin tosses, dice rolls and also many natural phenomenon such as average temperature.

Furthermore, if many independent effects contribute to an outcome, the resulting distribution is often well approximated by a normal distribution according to central limit theorem [1]. A common example is human height which clusters around an *average* with roughly symmetric variation.

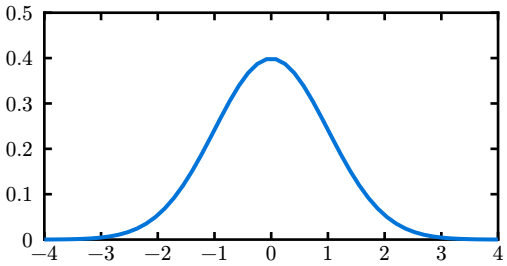

Figure 1: Normal distribution plot.

Contrary to this, there are systems which could not be described by normal distribution [2]. One example are the forest fire sizes [3]. If they were to adhere to normal distribution, there would be a known *average forest fire size*. Even though most forest fires are small, there is nothing limiting the potential size. This is one of the key properties of systems better described by the power law, scale-invariance.

Normal distribution is exponentially bounded, meaning that most of values are distributed within small range around the center (mean) and values far from it are very impropable. The probability density function of normal distribution with mean $\mu \in \mathbb{R}$ and variance $\sigma^2 > 0$ is

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/(2\sigma^2)}. \tag{1}$$

As opposed to this, the power law is *heavy tailed*. Its exponential character described in Equation (2) result in higher probability of extreme values. That corresponds with the scale-invariance: Any (forest fire) size is realistically possible, even though smaller sizes are more likely. Power law exhibit linear relationship between $\log f(x)$ and $\log x$. In a `log-log plot` it forms a straight line, as can be seen on Figure 2.

$$f(x) = ax^{-k}$$
$$k \ldots \text{constant exponent} \tag{2}$$

The emergence of the power law distribution is often connected to systems critical state. Consider phase transition of water from liquid to vapor. At the liquid-vapour boundary curve, both liquid and vapour can coexist but are distinctly separate. The boundary terminates at some critical temperature and critical pressure at which the critical point of a system is defined. While the system is in the critical state, the water is not in a single state.

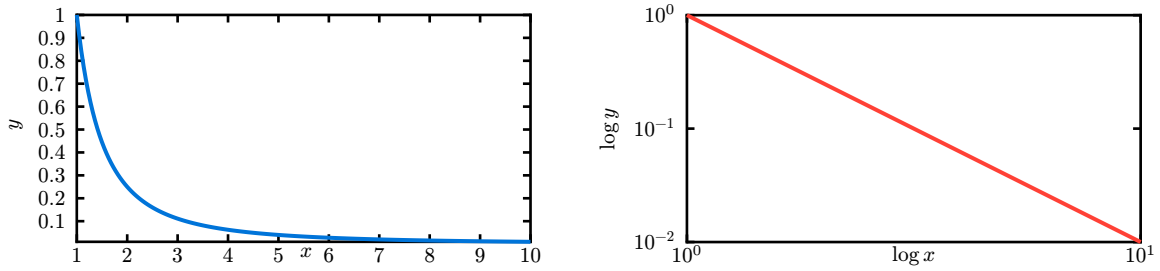

Figure 2: Function $y = x^{-2}$ plotted on linear scale (left) and on log-log scale (right) on interval $\langle 1, 10 \rangle$.

Systems in critical state are most responsive to input and behave unpredictably [4]. This is the case for phase transition and forest fires. However, there is a significant difference. Criticality can be reached in many systems by fine-tuning certain parameters. In the case of phase transition the parameters are pressure and temperature. However, there exist systems which reach criticality independently of parameters.

The phenomenon of systems reaching criticality by themselves is called *self-organized criticality (SOC)*. The concept of SOC was first discovered in 1987 by Bak, Tang and Wiesenfeld and its properties were demonstrated on a sandpile model [5]. The original model was a cellular automaton on a square and cubic lattice.

In this thesis, we investigate the behaviour of the sandpile model on different graph structures. Specifically, on small-world networks which are a class of random networks. Before this generalization we introduce few necessary concepts from graph theory.

2. Graph Theory

An undirected graph is defined as a pair, containing set of vertices and set of edges. The graph G has n vertices and m edges,

$$\begin{aligned} &G = (V, E) \\ &V = \{v_1, v_2, \ldots, v_n\} \\ &E \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}, \quad |E| = m. \end{aligned} \tag{3}$$

The neighbourhood $\mathcal{N}$ of a vertex v is a set of vertices connected to it via an edge,

$$\mathcal{N}(v) = \{u \mid \{u, v\} \in E\}. \tag{4}$$

The vertex degree $d(v)$ is defined as the size of vertex neighbourhood,

$$d(v) = |\mathcal{N}(v)|. \tag{5}$$

Path between two vertices u and v is a sequence of vertices beginning in the first and ending in the second where every two vertices in path must be directly connected,

$$P_{u,v} = (v_1, v_2, ..., v_k), \quad v_1 = u, \quad v_k = v, \\ \{v_l, v_{l+1}\} \in E \quad \forall l \in \{1, ..., k-1\} \ , \tag{6}$$

if there exist no such path, let $P_{u,v} = ()$.

Similarly, the path length is the number of *steps* (edges) needed to walk the path,

$$\text{len}\big(P_{u,v}\big) = \begin{cases} \big|P_{u,v}\big| - 1 & \big|P_{u,v}\big| \geq 2, \\ 0 & \text{else}, \end{cases} \tag{7}$$

if a path doesn't exist we define its legth as zero.

The shortest path between vertices u and v is the path with minimal length among all paths connecting these vertices. Let $\mathbb{P}(u, v)$ be the set of all paths between u and v,

$$P^*_{u,v} = \min_{P \in \mathbb{P}(u,v)} \text{len}(P). \tag{8}$$

A graph has an average (shortest) path length,

$$\text{len}(G) = \frac{1}{n(n-1)} \Sigma_{u \neq v} \ \text{len}\big(P^*_{u,v}\big). \tag{9}$$

We say that two vertices u, v are connected if at least one path between them exist, i.e.

$$\text{len}\big(P_{u,v}\big) > 0. \tag{10}$$

The graph is connected if all its vertices are connected,

$$\forall u, v \in V, u \neq v : \quad \text{len}\big(P_{u,v}\big) > 0. \tag{11}$$

A subgraph G' of graph G contains a subset of its vertices and edges,

$$G'(V', E') \subseteq G(V, E) \Leftrightarrow V' \subseteq V \wedge E' \subseteq E. \tag{12}$$

The graph component is a connected subgraph which is not part of any larger connected subgraph.

Complete graph K_n is a graph with n vertices and $\frac{n(n-1)}{2}$ edges. This means that every two vertices are directly connected and the average path is one.

The measurement of how much do individual vertices in graph cluster together is called global clustering coefficient and it is based on triplets of nodes. One triplet is defined as three vertices with two or three edges between them. If the number of mutual edges is two, the triplet is called open, if it is three, the triplet is called closed. The global clustering coefficient of graph $C(G)$ is defined as

$$C(G) = \frac{\#\text{ closed triplets}}{\#\text{ all triplets}}. \tag{13}$$

Small world network is a graph with certain characteristic [6]. It has high clustering coefficient and low distances. One example of small world network might be a human relationships network. The low distances might be expressed with respect to number of vertices as

$$\text{len}(G) \propto \log(|V|). \tag{14}$$

Small world networks tend to contain complete subgraphs (cliques). This is direct result of high clustering coefficient. Additionally, because the average path is small, they often contain *hubs* (vertices with high degree) which serves as connections between other vertices.

One of methods to quantify the *small-worldness* of a network is a small-world measure (ω) defined as follows [7]

$$\omega(G) = \frac{\text{len}(R)}{\text{len}(G)} - \frac{C(G)}{C(L)}. \tag{15}$$

Where G is the analyzed network, R is an equivalent random network to G, and L is an equivalent (ring) lattice network to G. This method aims to quantify the formulating properties of small-world network: high clustering coefficient which is *ideal* in lattice networks, and low average path which is *ideal* in random networks.

TODO: priklad, visual + vypocet

3. Sandpile Model

The very original sandpile model is a cellular automaton on a lattice. Every cell can hold up to K grains, the height of a cell is a function of position $z(x, y, ...)$. Fixed boundary conditions are used, on a boundary $z = 0$ cannot be changed. In two dimensions z is updated as follows:

$$\begin{aligned} z(x, y) &\to z(x, y) - 4 \\ z(x \pm 1, y) &\to z(x \pm 1, y) + 1 \\ z(x, y \pm 1) &\to z(x, y \pm 1) + 1 \end{aligned} \tag{16}$$

if z exceeds (or meets) critical value $K = 4$. The system is initialized randomly but with $z \gg K$. That initial state or rather the size of a lattice along with Z (the height of all cells) is called *configuration*.

For simplicity we can imagine a simple chessboard and falling sand grains. Each grain falls on a chess square determined by a random distribution[1]. If the number of grains on any square reaches four, the sandpile topples and the four grains fall on adjacent squares, possibly causing additional squares to topple. In case the square does not have four adjacent squares (i.e. it is on the edge of chessboard) the remaining grains fall off the board and are therefore removed from the model.

We say that cell (square) is stable if it contains less than K grains. The system is stable if all of its cells are stable. If a grain is added to any stable cell, it can destabilize it resulting in an avalanche. Avalanche is the set of topplings which occur before the system is stabilized again.

The sandpile model is a dissipative dynamical system. The dissipation is demonstrating on the boundaries. When a grain leaves the chessboard (or enters a boundary cell) it dissipated from the system. Dissipation is an important property of the model. Without dissipation the model supersaturates, leading to one infinite avalanche.

We can express the square lattice (chessboard) as a graph $G = (V, E)$ with the size given by[2] $\sqrt{n}$. For a boundary condition, the boundary vertices height is fixed to zero $z = 0$. Alternatively, we may imagine only one vertex, called *sink vertex*, with such condition. This sink is connected to all boundary vertices, thus creating the same effect. Please note, that this simplification creates a multigraph, because corner vertices and sink are connected via two edges. For the rest of this thesis we will consider the simplified version with one sink vertex and therefore a multigraph. However, it is trivial to create multiple sink vertices to restore a simple undirected graph.

[1] In the original model, all sand grains were distributed initially. It might be more obvious if we let grains fall one at a time for demonstration purposes.

[2] In a square lattice, the size$^2 = n$, which is the number of vertices in graph.

Figure 3: Comparison of square lattice sink vertices (blue). On the left there is a graph with sinks at the boundary - as in the original model. On the right there is a multigraph, the square lattice with a single sink connected to all vertices at the boundary which is hinted by outgoing edges.

All properties of the cellular automaton system holds on a square lattice. This model was later generalized from square lattice to arbitrary graph [8]. That generalization is important because it allows us to study the model's behaviour for different graph topologies.

The classical model has the following important property. The final configuration is independent of the order in which avalanches happen. This also implies that when adding two grains on different cells, the final configuration does not depend on the order of adding grains. It is because the addition of a sand grain can be seen as an operator, and these operators were proved to form an Abelian group [9].

This property was later shown to hold on finite (un)directed graphs [8], allowing us to study the behaviour of the model on different graph topologies, rather than studying the consequences of changing a toppling order.

3.1. Graph Topologies

Standard choice for the sandpile model graph is the square lattice with arbitrary size. This however does not realistically represent the model from the motivation story in Section 1 and other possible applications. That is why we introduce different graph topologies.

Before leaving square lattice entirely, we discuss toppling and dissipation rules on it. The original model presents a toppling rule described in Equation (16) and $K = 4$, where K is the number of grains which cause a single cell (vertex) to topple.

Dissipation rules on a general graph are more complex and further discussed later in Section 3.2. While the different definitions of K and toppling rules might be more deeply explored, we will only consider K being a function of a vertex,

$$K(v) = d(v), \quad v \in V. \tag{17}$$

The toppling rule is also expressed as a function of a vertex. Similarily to the original definition as cellular automaton, the height of a vertex is expressed as a function

$$\begin{aligned} z(v) &\to z(v) - K(v) \\ z(n) &\to z(n) + 1, \quad n \in \mathcal{N}(v). \end{aligned} \tag{18}$$

This simple rule states, that during the toppling process the original vertex loses as many grains as many neighbours it has and all its neighbours gain one. The benefit of this K and toppling rule is their simplicity.

For example, it might be interesting to set $K = \text{const}$, but that would create problems. If a vertex has less neighbours, it is unclear where the additional grains go during a topple. If it has more neighbours, does it artifically generate more grains, or distribute fewer grains between randomly selected neighbours?

Note that in these reasonings, the neighbourhood of a vertex $\mathcal{N}(v)$ contains sink vertex, and possibly multiple times. The toppling rule will not work if $K(v) = 0$. We acknowledge this constraint but leave its resolution to dissipation rules.

With K, toppling rule and dissipation rules specified, we now continue onto the properties of different graph topologies. Square lattice is a regular graph, meaning the degree of all vertices is the same [10], if we consider sink vertex.

Regularity is not often found in human social networks, meaning graphs where vertices represent individuals and edges their mutual acquaintance. Purely random graph also would not be optimal since it is not regular. However, human networks are not completely random. Fortunately, a small-world network (better described in Section 2) are almost perfect.

The three most known random graph generators are the Erdős–Rényi (ER) model [11], Barabási-Albert (BA) [12] model and Watts-Strogatz (WS) model [6]. Each of these methods generates the graph differently resulting in various *small-worldness*.

The **ER model** was the first discovered. There are two variants, both of which start with graph with n vertices. The $G(n, m)$ model chooses a graph uniformly from the set of all possible graphs with n vertices and m edges. The $G(n, p)$ model creates all possible edges but only with the probability p.

The $G(n, m)$ model ensures that any edge is equally likely and the number of edges is known before. This, however, does not provide the resulting graph being connected. The $G(n, p)$ model ensures that every edge has a fixed probability of being present, also not ensuring connected graph.

Another drawback of this model is the low clustering coefficient. That is a consequence of all edges having the same probability, which is not usually seen in social networks and in turn in small-world networks.

The latest to be discovered is the **BA model**. The important observation BA made is that many observed networks are *scale-free*, meaning they have power-law degree distribution. Their method of generating has two key concepts: growth and preferential attachment.

Growth simply means that the graph is build up from small number of vertices. Preferential attachment means for a vertex, the higher degree it has, the more likely it is to be connected to new vertex.

The algorithm for generating the graph is, simply put, a loop in which one vertex is added to the graph and connected m vertices. They are chosen from all vertices with probability

$$p(v) = \frac{d(v)}{\Sigma_u d(u)}, \quad u, v \in V \ , \tag{19}$$

where v is the examined vertex and u represents all vertices. The newly added vertex is yet not considered.

The algorithm takes G_0, n, m where m is the number of edges for each new vertex and n represents the number of vertices in final graph. The first arguments is a starting graph with at least m vertices. In practice, G_0 is usually chosen as a complete graph K_m of size m.

This algorithm yields a quick emergence of hubs, vertices with high degree. BA provides example i.e., newly create webpage is more likely to link to well-known websites which already have high degree.

As the consequence of hubs' existence, the average path is short. However, interesting property of BA model is the dependance of clustering C on $N = |V|$,

$$C \sim \frac{\ln(N^2)}{N}. \tag{20}$$

This behaviour is different from small-world network where clustering is independent of size [12].

This model generates a connected graph, because when new vertex is added, it is connected to exactly m existing vertices.

In the **WS model**, authors have also defined the small-world measure (ω) discussed in Equation (15). WS argued that ER model lacks two important properties often observed in small-world networks: high clustering coeficient and the convergence of degree distribution to power law. Their model was designed to address the first issue.

The generation algorithm is simple: 1. start with a regular ring lattice, 2. rewire every edge with probability β. The idea is to preserve the short average path present in ER model, but simultaneously create high clustering.

This is done by interpolating between the regular ring lattice (which has high clustering coefficient) and ER model. When $\beta = 0$, no edge is rewired and the ring lattice holds. It cannot approach the ER model, because rewiring does not remove edges and therefore the graph is connected - not a guaranteed property in ER model.

The algorithm for WS graph generation needs these parameters: number of vertices N, mean degree K (even integer) and rewiring probability β. When $\beta = 1$ it approaches a structure close to ER $G(n, p)$ model with $p = \frac{K}{N-1}$.

One necessary constraint on parameters is that $N > K$. It is widely agreed upon to use $N \gg K$ (e.g. $N = 10K$) when constructing graph.

While this method was designed to have high clustering coefficient and low average path length, the degree distribution is not power law as seen in BA model. This divergence from observable small-world networks (e.g. airport networks) is reflected in all vertices having similar degree.

Either of these methods of generating small-world network has its imperfections. In the interactive tool developed for the needs of this thesis are implemented all mentioned methods. Note, that from these only WS method focus solely on small-worldness, BA method is focused on scale-free networks and ER method on random graphs.

3.2. Dissipation Rules

With the introduction of arbitrary graph we must reconsider dissipation rules. The square lattice has explicit boundaries which serve as sinks. However, many graphs do not have an implicit boundary. The existence of sink is fundamental for the model not to supersaturate.

A dissipation rule determines how many times a vertex should be connected to the sink. There is not a single rule which would be generally applied to any graph. We discuss different rules, their properties and analogies. Dissipation rules are not well-known thus their names are created for the purposes of this thesis.

We can present a dissipation rule as a function of vertex returning an integer which denotes how many times a vertex should be connected to the sink.

$$D : v \to \mathbb{N} \quad . \tag{21}$$

For example, the graph representation of the original model uses a **Fill to Four** rule. It states that the degree of any vertex must be four or more. The sink vertex is connected to all vertices as many times it is needed to fulfil that rule. For the square lattice, only vertices on the edge are connected (exactly once except for corner vertices, which are connected twice).

$$D(v) = \begin{cases} 4 - d(v) & d(v) \leq 4, \\ 0\ . \end{cases} \tag{22}$$

For different graphs though, this rule might be insufficient. It does not ensure dissipation. This rule does not consider the vertex degree if it is higher than four.

To ensure dissipation on any graph, the rule must connect all vertices to the sink at least once. That is because there might exist vertices which are not connected to any other vertex. A problem arises with such vertex if it were not connected: $d(v) = 0 = K$ meaning it should trigger an avalanche but that itself is nonsensical. Dissipation rules must prevent this from happening.

If we consider connected graphs the dissipation condition softens. It is guaranteed that for all pairs of vertices exist at least one path therefore only one sink is sufficient to ensure dissipation. This might be useful for the sandpile model is only interested on connected graphs where they exhibit critical behaviour.

One possible rule which guarantees dissipation on any graph is **All Once**. It ensures just the necessary condition that every vertex must be connected to sink at least once but nothing more. The rule is fair to all vertices as the connection to the sink is not exclusive to vertices of some characteristic[3].

$$D(v) = 1 \quad . \tag{23}$$

One rule sufficient for connected graph is **As Many As Neighbours**. This rule connects every vertex to sink proportionally to its neighbourhood size. When considering connected graph this rule ensures dissipation and even provide a connection to the sink to all vertices.

$$D(v) = d(v) \quad . \tag{24}$$

The *Fill to four* rule may be seen as a particular case of the general **Fill to N** rule. The specific case where $N = 4$ is useful for (two dimensional) square lattice, but for three dimensional lattice, for example, is more adept $N = 6$. The generalized rule with static value (e.g. 4 or 6) might be central for particular graph topologies (e.g. the lattice).

However, if we permit N to vary based on specific graph parameters, we introduce **Fill up** rule, where N is equal to the degree of vertex with largest neighbourhood.

$$N = \max_{v \in V} d(v) \tag{25}$$

Static N does not ensure dissipation on general graph, because it might only contain vertices with degree higher or equal N. Contrary to that, dynamic D ensures dissipation on any graph with at least one edge and two vertices, with rule being:

$$D(v) = \begin{cases} N - d(v) & d(v) \leq N\ , \\ 0. \end{cases} \tag{26}$$

Every rule represents a different approach which might not be applicable to all graph topologies. Below are listed necessary requirements on graph for all dissipation rules in order to ensure dissipation:

Fill to N Each graph component has at least one vertex with degree $d(v) < N$.
All once No requirement.
As many as neighbours There is no vertex with degree zero, i.e. the graph is connected.
Fill up At least two vertices and one edge exist.

[3]Possible characteristics include vertex neighbourhood size, shortest path by which it reaches all other vertices and more.

Different graph topologies are suitable for various subset of dissipation rules, as shown in Table 1. Note that specific graph generated by any random method may be suitable for additional dissipation rules, the table covers the general case.

	Square lattice	**ER**		**BA**	**WS**
	$G(n)$	$G(n,p)$	$G(n,m)$	$G(G_0,n,m)$	$G(N,K,\beta)$
Fill to N	$N \geq 4$	✘	✘	✘	✘
All once	✓	✓	✓	✓	✓
As many as neighbours	✓	✘	✘	$m > 1$	✓
Fill up	✓	$p = 1$	$m \geq 1$	✓	✓

Table 1: The suitabilty comparison for different dissipation rules on graph topologies. Rule is suitable (✓), not suitable (✘) or only suitable for graphs with extra constraints.

It might seem as *Fill to N* rule should be suitable for more dissipation rules, but the parameter N is set to be static and thus cannot be set based on graph properties. Standard constraints discussed in Section 3.1 are put on BA and WS models.

4. Observed Behaviour

TODO: how did it work for different types of graphs and dissipation rules? also the graph configurations (eg β for WS)

TODO: metrics, how it changed criticality? origin distribution, avalanche size (max, avg)

TODO: imagery from the app

5. Conclusion

TODO

6. Technical Documentation

The accompanying interactive tool for this thesis allows users to simulate SOC on sandpile model with all discussed graph topologies and dissipation rules. It serves primarily as a visualization of the problem with simple statistics alongside it.

The technologies used were the programming language **C++23**, with libraries *igraph*, *Dear ImGui*, *SDL3* and *OpenGL*. For compilation was used *CMake* with *Clang* or *GCC*. Libraries are managed via *vcpkg* or as *Git* submodules. *Backward-cpp* was used for beautiful stack trace. The whole project is hosted on *GitHub*.

Naming convention:

Classes, structs, enums Pascal Snake Case
Example: `Square_Lattice`

Variables, parameters, functions, methods, namespaces snake_case
Example: `sink_rule`

Private members (or to avoid name clashes) Trailing underscore
Example: `avalanche_sizes_`

The application is a visual tool and as such have an infinite render loop. The main logic is inside *App* class. This class run the main loop and using various functions in *ui* namespace renders the right parts

of the ui. Note that *Dear ImGui* is an immediate mode GUI thus the state of UI is directly obtained from application state every time the UI is rendered.

As the application allows user to select any combination of graph topology and dissipation rule, the state is stored in structure *Sim_Config*. The selected visualization method is held by *Vis_Config*.

For the graph representation serves the *Graph* class, which stores all vertices and edges in offset and neighbours list representation, also called compressed adjacency list. That representation is fitting as no graph manipulation happens during the simulation. The Graph also holds the sand height for simulation and vertex positions for visualization.

For constructing of the graph was used the *igraph* library which already implements all discussed graph topologies, however the sink is added manually based on the specific dissipation rule.

The implementation aims to be library-independent, therefore if *igraph* were to be replaced, only the generating part have to be rewritten. Similarly, when extending the app of another graph generating library, only look inside `graph/Generator.hpp` and `graph/Converter.hpp`.

The statistics are collected using event listeners in class *Stats_Collector*. There are only three events, however that could be extended if desired.

The window the application is viewed in is created via *SDL* and *OpenGL*, the minimal implementation is in `ssoc::ui::Window_Context`.

Bibliography

[1] R. Durrett, *Probability: Theory and Examples*, 5th ed. in Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, 2019.

[2] A. Clauset, C. R. Shalizi, and M. E. J. Newman, "Power-Law Distributions in Empirical Data," *SIAM Review*, vol. 51, no. 4, pp. 661–703, 2009, doi: 10.1137/070710111.

[3] B. D. Malamud, G. Morein, and D. L. Turcotte, "Forest Fires: An Example of Self-Organized Critical Behavior," *Science*, vol. 281, no. 5384, pp. 1840–1842, 1998, doi: 10.1126/science.281.5384.1840.

[4] P. Bak, *How Nature Works: The Science of Self-Organized Criticality*, 1st ed. New York, NY: Copernicus, 1996, p. XIII, 212. doi: 10.1007/978-1-4757-5426-1.

[5] P. Bak, C. Tang, and K. Wiesenfeld, "Self-organized criticality: An explanation of the 1/f noise," *Physical Review Letters*, vol. 59, no. 4, pp. 381–384, July 1987, doi: 10.1103/PhysRevLett.59.381.

[6] D. J. Watts and S. H. Strogatz, "Collective dynamics of ``small-world'' networks," *Nature*, vol. 393, no. 6684, pp. 440–442, 1998, doi: 10.1038/30918.

[7] Q. K. Telesford, K. E. Joyce, S. Hayasaka, J. H. Burdette, and P. J. Laurienti, "The ubiquity of small-world networks." [Online]. Available: https://arxiv.org/abs/1109.5454

[8] A. E. Holroyd, L. Levine, K. Mészáros, Y. Peres, J. Propp, and D. B. Wilson, "Chip-Firing and Rotor-Routing on Directed Graphs," in *In and Out of Equilibrium 2*, Birkhäuser Basel, 2008, pp. 331–364. doi: 10.1007/978-3-7643-8786-0_17.

[9] D. Dhar, "Self-Organized Critical State of Sandpile Automation Models," *Physical review letters*, vol. 64, pp. 1613–1616, 1990, doi: 10.1103/PhysRevLett.64.1613.

[10] W.-K. Chen, *Graph Theory and Its Engineering Applications*. Singapore; River Edge, NJ: World Scientific, 1997.

[11] P. L. Erdős and A. Rényi, "On random graphs. I.," *Publicationes Mathematicae Debrecen*, 2022, [Online]. Available: https://api.semanticscholar.org/CorpusID:253789267

[12] A.-L. Barabási and R. Albert, “Emergence of Scaling in Random Networks,” *Science*, vol. 286, no. 5439, pp. 509–512, Oct. 1999, doi: 10.1126/science.286.5439.509.